

pilot-live Technical Manual

Version 428e509

2026-09-30

Contents

1	Markdown Visualiser	5
1.1	Purpose	5
1.2	Technologies	7
1.3	Architecture at a Glance	7
1.4	Prerequisites	7
1.5	Local Development	8
1.6	Running Tests	8
1.7	Deployment	8
1.8	Documentation	8
1.9	Licence	9
2	Architecture	10
2.1	Overview	10
2.2	Components	10
2.3	Information Flow	11
2.4	External Systems	11
2.5	Infrastructure	11
2.6	Deployment Architecture	11
3	Design	13
3.1	Design Principles	13
3.2	Component Boundaries	13
3.3	Key Abstractions and Patterns	14
3.4	Configuration Strategy	14
3.5	Error Handling	14
3.6	Logging	15
3.7	Authentication and Authorisation	15
3.8	State Management	15
3.9	API Conventions	15
4	How It Works	16
4.1	Workflows	16
5	Data Model	19
5.1	Overview	19
5.2	Entities	19
5.3	Relationships	19
5.4	Constraints and Indexes	20
5.5	Ownership and Retention	20
5.6	Migrations	20
5.7	Business Rules	20

6	API	22
6.1	Overview	22
6.2	Endpoints	22
6.3	Errors	23
7	Security	24
7.1	Trust Boundaries	24
7.2	Authentication	24
7.3	Authorisation	24
7.4	Secrets Management	24
7.5	Session Handling	25
7.6	API Security	25
7.7	Network Exposure	25
7.8	Input Validation	25
7.9	Dependency Security	26
7.10	Logging and Audit	26
7.11	Sensitive Data	26
7.12	Rate Limiting	26
7.13	Encryption and TLS	26
7.14	Security Assumptions	26
7.15	Responsibilities	27
8	Deployment	28
8.1	Environments	28
8.2	Build	28
8.3	Deploy	29
8.4	Configuration	29
8.5	Rollback	29
9	Testing	30
9.1	Strategy	30
9.2	Unit Tests	30
9.3	Integration Tests	30
9.4	End-to-End Tests	31
9.5	Security Tests	31
9.6	Manual and Smoke Tests	31
9.7	Test Environments and Data	31
9.8	Running Tests	31
9.9	Required Before Changes	32
10	Operations	33
10.1	Monitoring	33
10.2	Alerts	33
10.3	Routine Operations	33
10.4	Incident Response	34
11	Maintenance	35
11.1	Starting and Stopping	35
11.2	Dependency Updates	35
11.3	Database Migrations	35
11.4	Secret Rotation	36
11.5	Configuration Management	36

11.6	Logs	36
11.7	Backups	36
11.8	Releases	36
11.9	Making Production Changes	36
11.10	Known Technical Debt	37
11.11	Fragile Areas	37
12	Troubleshooting	39
12.1	Known Problems	39
13	Dependencies	42
13.1	Critical Dependencies	42
14	ADR-001: Single client-side markdown pipeline	45
14.1	Status	45
14.2	Context	45
14.3	Options Considered	45
14.4	Decision	45
14.5	Rationale	45
14.6	Consequences	46
14.7	Related Components	46
15	ADR-002: Sanitiser placed immediately after rehype-raw	47
15.1	Status	47
15.2	Context	47
15.3	Options Considered	47
15.4	Decision	47
15.5	Rationale	47
15.6	Consequences	48
15.7	Related Components	48
16	ADR-003: Contained, unauthenticated print service	49
16.1	Status	49
16.2	Context	49
16.3	Options Considered	49
16.4	Decision	49
16.5	Rationale	49
16.6	Consequences	50
16.7	Related Components	50
17	ADR-004: Composite string sort key for reading order	51
17.1	Status	51
17.2	Context	51
17.3	Options Considered	51
17.4	Decision	51
17.5	Rationale	51
17.6	Consequences	51
17.7	Related Components	52
18	ADR-005: Repair markdown for rendering only, and report every re-pair	53
18.1	Status	53
18.2	Context	53

18.3 Options Considered	53
18.4 Decision	53
18.5 Rationale	53
18.6 Consequences	54
18.7 Related Components	54
19 ADR-006: Use webkitGetAsEntry for folder drops	55
19.1 Status	55
19.2 Context	55
19.3 Options Considered	55
19.4 Decision	55
19.5 Rationale	55
19.6 Consequences	55
19.7 Related Components	56
20 ADR-007: Official Playwright runtime image, run as non-root	57
20.1 Status	57
20.2 Context	57
20.3 Options Considered	57
20.4 Decision	57
20.5 Rationale	57
20.6 Consequences	57
20.7 Related Components	58
21 ADR-008: Self-hosted fonts and no third-party requests	59
21.1 Status	59
21.2 Context	59
21.3 Options Considered	59
21.4 Decision	59
21.5 Rationale	59
21.6 Consequences	59
21.7 Related Components	60
22 Changelog	61
22.1 [Unreleased]	61

Chapter 1

Markdown Visualiser

Drop a folder of markdown files into a browser and get back a typeset reading edition: ordered, cross-linked, and set on an archival paper surface built for reading rather than for scanning. Export it as a print-quality PDF or as a single self-contained HTML file.

1.0.1 Design by Meng To

The entire visual design of this project comes from the work of [Meng To](#) — full credit for how this looks belongs to him. The archival-paper direction, the serif typographic hierarchy, the index-as-navigation, the layered elevation and the progress rail are all implementations of design direction he published in [MengTo/Skills](#) (MIT).

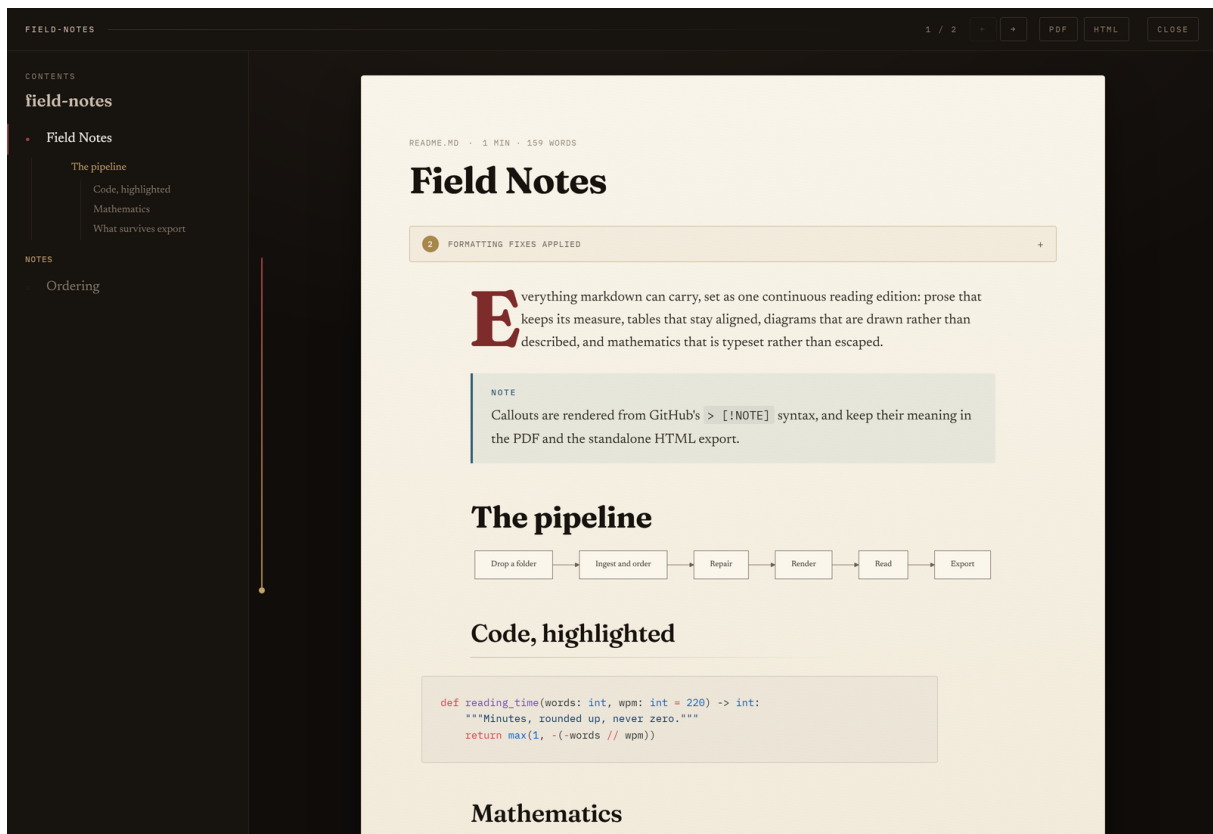
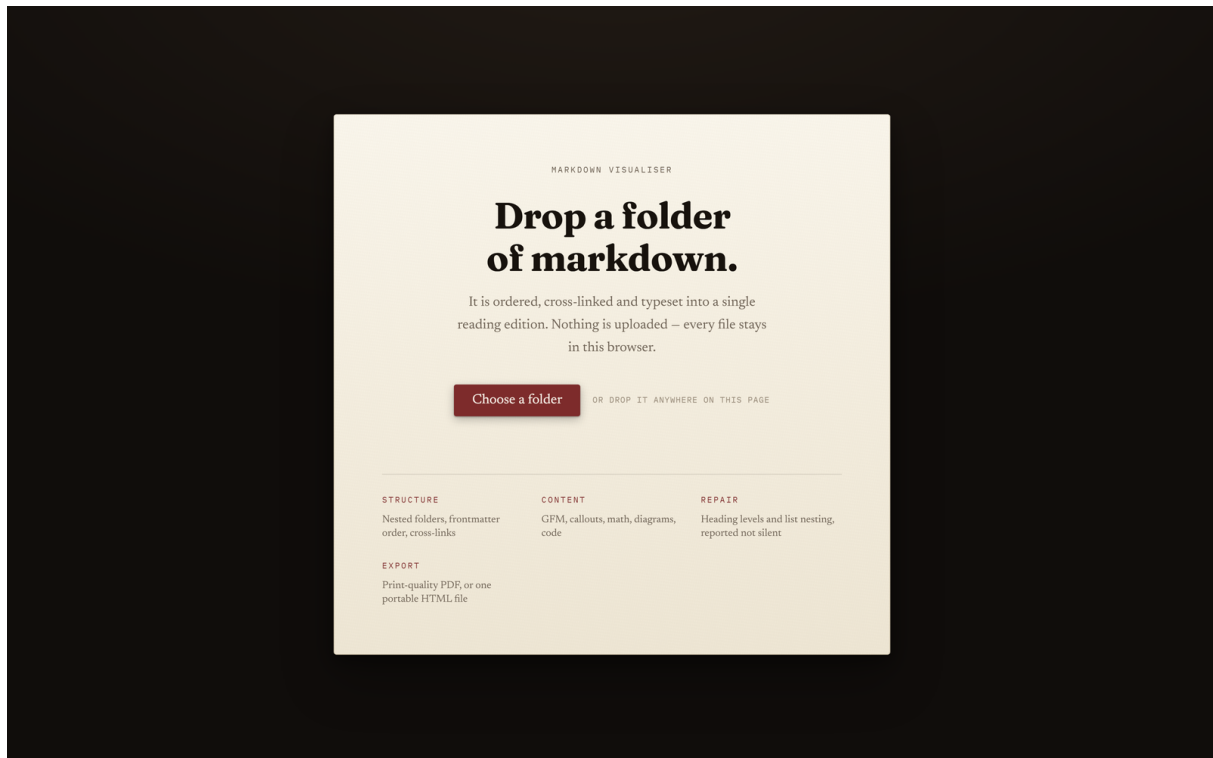
See CREDITS.md for exactly which skills shaped which parts, and go look at [his work](#) and [Design+Code](#) directly.

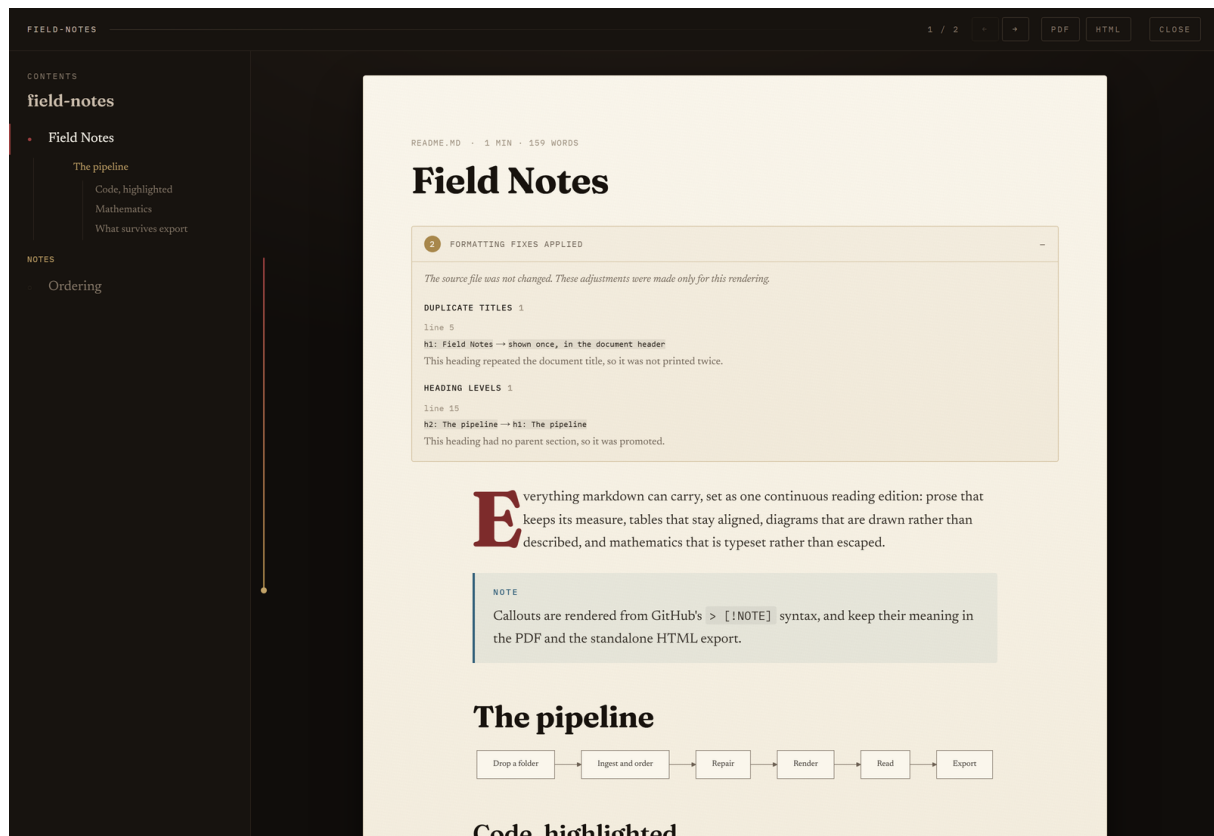
1.1 Purpose

It exists for one narrow purpose — helping people read markdown files in a neater, more legible way than a raw editor or a utilitarian preview pane allows.

- Understands a folder: nested directories, frontmatter ordering, numeric file-name prefixes, README/index conventions, and relative links that become in-app navigation.
- Renders GFM, > [!NOTE] callouts, KaTeX maths, Shiki-highlighted code, Mermaid diagrams and local images with a lightbox.
- Repairs messy markdown for the rendering only, and lists every repair (line, before, after) in the document. The source file is never modified.
- Exports a PDF with running heads and page numbers, or one portable HTML file that works offline.

Drops are capped at 200 MB / 2000 files / 10 MB per file; PDF request bodies at 20 MB. Not in scope: editing markdown, persistence between sessions, authentication, cloud sync, or live filesystem watching.





1.2 Technologies

TypeScript (strict), React 19, Vite 7, Zustand, unified/remark/rehype, Shiki, KaTeX, Mermaid, Fastify 5, Playwright (Chromium), Vitest, Docker. Typography is Fraunces, Newsreader and IBM Plex Mono (SIL OFL), self-hosted, so the app makes zero third-party requests. See docs/DEPENDENCIES.md.

1.3 Architecture at a Glance

There is exactly one markdown pipeline, and it runs in the browser. The server is a static host plus a stateless print service that receives already-rendered HTML.

```

browser:  drop folder → ingest → DocumentSet → repair → sanitise → render → reader
                                                ↓
                                                inline all CSS + assets
                                                | POST rendered HTML (PDF only)
container: Fastify → Playwright Chromium (JS off, network blocked) → PDF
  
```

Trade-off: document content leaves the browser at PDF export time. It is held in memory for one request and never written to disk. The HTML export is entirely local.

Details: docs/ARCHITECTURE.md, docs/DESIGN.md, docs/SECURITY.md.

1.4 Prerequisites

- Node.js 20 or later (package.json engines; init.sh enforces it) and npm.

- Docker, for the container and for the browser and security tests.

1.5 Local Development

```
./init.sh # checks toolchain, npm ci, vendors design skills, typ
docker compose up --build # → http://localhost:8080
```

For development without Docker: `npm run dev` (Vite) and `npm run dev:server` (`tsx watch server/index.ts`). Whether PDF export works from the Vite dev server without the container: uncertain — verify with developer.

1.6 Running Tests

```
npm run typecheck
npm run lint
npm test -- --run
npm run test:e2e # browser drives; needs the container running
npm run test:security # adversarial probe of the PDF endpoint
```

See docs/TESTING.md.

1.7 Deployment

Docker two-stage build, run with `docker compose up --build -d`. Bound to `127.0.0.1:8080` because the service is unauthenticated. The application is stateless. See docs/DEPLOYMENT.md and docs/OPERATIONS.md.

1.8 Documentation

Document	Contents
docs/ARCHITECTURE.md	Components and information flow
docs/DESIGN.md	Boundaries, conventions, error handling
docs/HOW_IT_WORKS.md	End-to-end workflows
docs/DATA_MODEL.md	In-memory entities, limits, business rules
docs/API.md	The two HTTP endpoints
docs/SECURITY.md	Trust boundaries and controls
docs/DEPLOYMENT.md	Build, configuration, rollback
docs/TESTING.md	Test strategy and gates
docs/OPERATIONS.md	Monitoring and incidents
docs/MAINTENANCE.md	Handover, debt, fragile areas
docs/TROUBLESHOOTING.md	Known problems
docs/DEPENDENCIES.md	Critical dependencies
docs/adr/	Architecture decision records
CHANGELOG.md	Changes

1.9 Licence

MIT — see LICENSE. The design credit above is not a licence condition but a statement of fact: the look of this application is Meng To's work, and CREDITS.md records it in full. Bundled fonts are SIL OFL 1.1.

Chapter 2

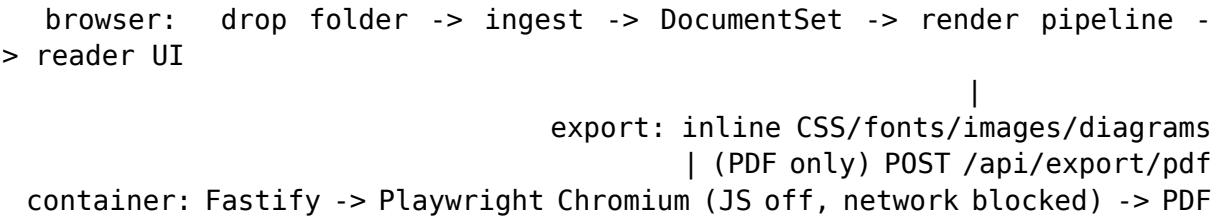
Architecture

Which components exist and how information flows between them.

2.1 Overview

Markdown Visualiser turns a dropped folder of markdown files into a typeset reading edition in the browser. It can export that edition as one self-contained HTML file or as a PDF.

There is one markdown pipeline and it runs in the browser. The server is a static file host plus a stateless PDF print service. It never sees markdown, only HTML the browser has already rendered. See ADR-001.



2.2 Components

Component	Location	Responsibility
Ingest	src/ingest/	Walks a dropped folder (
Domain types	src/types/domain.ts	DocumentSet, MarkdownDo
Render pipeline	src/pipeline/	The unified chain in rend
State	src/state/	store.ts is a Zustand sto
Reader UI	src/reader/, src/app/, src/ingest/Dropzone.tsx	React components: pape
Export	src/export/	standalone.ts builds the
Styles	src/styles/	Design tokens, paper sur
Web server	server/index.ts	Fastify: serves dist/, /he
Print service	server/pdf.ts	Shared Chromium, per-r

2.3 Information Flow

1. **Ingest.** The Dropzone passes a `DataTransfer` or `FileList` to the store. The walker collects files, `stripCommonRoot` removes the shared root folder, and `buildDocumentSet` produces a `DocumentSet`. Nothing leaves the browser.
2. **Render.** When a document is opened, `state/render.ts` calls `renderDocument`. The result is cached in a `WeakMap` keyed by set, and the next and previous documents are prefetched when the browser is idle.
3. **Display.** The reader injects the sanitised HTML and draws Mermaid diagrams lazily.
4. **HTML export.** `buildStandalone` renders every document off-screen, draws all diagrams, inlines images as data URIs and inlines the page's CSS. The result is one HTML string.
5. **PDF export.** The same string is POSTed as JSON to `/api/export/pdf`. Chromium loads it via `setContent`, and the service returns the PDF.

Details and failure modes are in `HOW_IT_WORKS.md`.

2.4 External Systems

- **Chromium via Playwright** (`playwright-core 1.62.1`). The runtime image is the official Playwright image `v1.62.1-noble`, which provides the browser. See `DEPENDENCIES.md`.
- **No third-party network calls at runtime.** Fonts are self-hosted, and the application makes no third-party requests (`README`, DoD 0.7). The reader does not call any other backend.
- **Build-time only:** `init.sh` clones `MengTo/Skills` from GitHub to vendor design skills into `.claude/skills/mengto/`. If the clone fails, the script continues.
- **Documentation tooling:** `.github/workflows/docforge.yml` runs `docforge`, which is installed from a GitHub repository. Its source is marked uncertain in the workflow itself.

2.5 Infrastructure

A single container built from `Dockerfile` and run by `docker-compose.yml`. It has no database, no cache service, no queue and no persistent volume. The only writable path is a 512 MB `/tmp tmpfs`. Limits: `mem_limit 2g`, `pids_limit 512`, all capabilities dropped, `no-new-privileges`, read-only root filesystem, `ipc: host`, `init: true`. The container runs as `pwuser`, not `root`.

No hosting platform, load balancer or orchestration is defined in the repository. Where production runs: uncertain — verify with developer.

2.6 Deployment Architecture

The `Dockerfile` has two stages. The build stage (Node 24 `bookworm-slim`) runs `npm ci`, `typecheck` and `npm run build`. The runtime stage (Playwright noble image) installs production dependencies only and copies `dist/` and `dist-server/`. It

listens on port 8080. Compose publishes it on 127.0.0.1:8080 only. Steps are in DEPLOYMENT.md.

Chapter 3

Design

Design intent: boundaries, abstractions and conventions.

3.1 Design Principles

- **One renderer, in the browser.** The server never parses markdown. The PDF matches the screen because it is printed from the HTML the screen showed. ADR-001
- **Contain, don't trust.** Untrusted input is sanitised at one named boundary, and the one server endpoint that handles hostile HTML is layered so no single control is load-bearing. ADR-002, ADR-003
- **Never change a document silently.** Every repair is recorded as a RepairNote and shown to the user. The source file is never modified. ADR-005
- **One bad file never costs the set.** Failures are recorded on the document (error) or in skipped, not thrown away or fatal.
- **Local first.** Nothing is uploaded. The one exception is PDF export, which sends rendered HTML to the server for one request and stores nothing.
- **Zero third-party requests,** so the HTML export works offline. ADR-008

3.2 Component Boundaries

- `ingest/` produces a `DocumentSet` and knows nothing about rendering. The `DocumentSet` is the only description of the user's files. The app is described as "a pure function of a `DocumentSet`" in `domain.ts`.
- `pipeline/` turns one `MarkdownDoc` into a `RenderResult`. It has no UI dependencies. It receives assets and a link resolver through `RenderContext`.
- `state/` holds the current set and the render cache.
- `reader/` displays results and owns no markdown logic.
- `export/` serialises a set and calls the server.
- `server/` has no dependency on the pipeline. It has four runtime dependencies: `fastify`, `@fastify/static`, `p-limit` and `playwright-core`. `LIMITS.pdfBodyLimitBytes` in `src/types/domain.ts` is mirrored by a separate constant in `server/index.ts`, so the two must be changed together.

3.3 Key Abstractions and Patterns

- **DocumentSet / MarkdownDoc / RenderResult:** immutable data passed between layers. See DATA_MODEL.md.
- **Trust boundary.** rehype-sanitize runs directly after rehype-raw. KaTeX, Shiki, Mermaid placeholders and asset rewriting run after it, on an already-clean tree. Nothing user-controlled may be added below it. sanitize.ts therefore allows only *marker* classes, not *style*, `<svg>` or KaTeX classes.
- **Composite sort key.** Reading order is a single dot-joined string compared as text: `dirRank.explicitOrder.indexRank.numericPrefix.naturalName`. ADR-004
- **Repair pass as a remark plugin.** It runs on the mdast tree before conversion to HTML and pushes RepairNotes into a caller-owned sink. Five rules exist: heading-skip, duplicate-h1, list-nesting, mixed-markers, unbalanced-emphasis. Some are only reported. For example, unbalanced emphasis is reported and left as literal text.
- **Object URLs for assets,** owned by the set and revoked when the set is discarded. They are converted to data URIs only at export time.
- **Render cache.** WeakMap per set plus an in-flight map, so concurrent requests for the same document share one render. Nothing invalidates it, because results cannot change for a given set.
- **Folder walking** uses `webkitGetAsEntry`. ADR-006

3.4 Configuration Strategy

Server configuration is by environment variable only. PORT (default 8080), HOST (default 0.0.0.0), STATIC_ROOT (default `../dist` relative to the bundle) and LOG_LEVEL (default `info`). The Dockerfile sets NODE_ENV, PORT, HOST and STATIC_ROOT; Compose sets NODE_ENV and LOG_LEVEL. Limits are constants in code, not configuration: file and size caps in LIMITS (`src/types/domain.ts`), and print-service timeouts and concurrency in `server/pdf.ts`. There are no config files or secrets. See DEPLOYMENT.md.

3.5 Error Handling

- **Ingest:** IngestError with a code (`too-large`, `too-many-files`, `no-markdown`, `read-failed`) and a user-readable message that includes measured sizes. The store maps any error to an error phase.
- **Per file:** an unreadable file becomes a MarkdownDoc with error set and still appears in the index. Malformed frontmatter YAML is left in the body as content.
- **Render:** a document with error renders as empty. A render failure is shown as a failed state for that document only.
- **Server:** 400 for a bad body, 429 with `retry-after: 5` when busy, 504 on timeout, 503 for anything else. The 503 body is generic and the detail goes only to the log, because browser errors can disclose paths. See API.md.
- Unknown or unreadable links are classified missing rather than rendered as dead links.

3.6 Logging

Only the server logs. It uses Fastify's built-in logger (Pino), with the level set by LOG_LEVEL. Startup failure and shutdown signals are logged, and render failures are logged with the error at error level. The client has no logging framework: uncertain — verify with developer. See OPERATIONS.md.

3.7 Authentication and Authorisation

None, by design. The app holds no data and grants no capability beyond printing HTML the caller supplied. Protection is network placement: Compose binds to 127.0.0.1. See SECURITY.md and ADR-003.

3.8 State Management

- **Client:** a Zustand store (useReader) with phase, set and activeDocId. Replacing or resetting the set revokes its object URLs.
- **Server:** stateless. The only shared state is one Chromium Browser, which is relaunched if it disconnects, and a p-limit queue. Each request uses a fresh BrowserContext.
- **Persistence:** none. A page reload loses the set.

3.9 API Conventions

The API is two endpoints with JSON errors of the form { "error": "<message>" }. Anything under /api/ that is not found returns a JSON 404. Everything else falls back to index.html. Responses carry x-content-type-options: nosniff, referrer-policy: no-referrer and cross-origin-opener-policy: same-origin. Hashed assets are cached as immutable, and index.html is no-cache. See API.md.

Chapter 4

How It Works

End-to-end workflows. For each: trigger, receiver, processing, services called, storage, async work, result, failure modes.

4.1 Workflows

4.1.1 Workflow: Drop a folder and build a document set

1. Trigger: the user drops a folder or files on the page. A directory picker (webkitdirectory file list) is the fallback.
2. Received by: Dropzone calls `useReader.ingestDataTransfer` or `ingest-FileList (src/state/store.ts)`.
3. Processing: `walkDataTransfer` captures entries synchronously (the item list is emptied when the handler returns). It then walks directories, draining `readEntries` in a loop, and skips `.git`, `node_modules`, `dist`, `build` and similar. `stripCommonRoot` removes a single shared root folder. `buildDocumentSet` checks limits first (2000 files, 200 MB total). Files are then classified: `.md/.markdown/.mdown/.mkd` are documents, common image types become assets, and everything else goes into `skipped`. Markdown files over 10 MB are skipped. For each document, frontmatter is parsed. The title is `frontmatter.title`, else the first H1, else a prettified filename. `order` is read from frontmatter. Documents are sorted by the composite key.
4. Services called: none.
5. Data stored: in memory only (Zustand store). Images become object URLs.
6. Asynchronous work: file reads, and the phase transitions `reading` → `building` → `ready`.
7. Result: the first document in reading order becomes active.
8. What can fail: `IngestError` for too many files, too large, no markdown found, or read failure. The store shows the message. A file that cannot be read appears in the index marked as failed. Dropping a very large tree may stop at 2000 files, because the walker stops there. Malformed frontmatter is treated as plain content.

4.1.2 Workflow: Render and read a document

1. Trigger: a document becomes active (open, index click, keyboard navigation).

2. Received by: `useRenderedDoc` in `src/state/render.ts`, which calls `render-Document` (`src/pipeline/render.ts`).
3. Processing: a discovery parse finds the code languages so Shiki can load only those grammars. The main chain is: `parse` → `frontmatter` → `GFM` → `math` → **repair** → `callouts` → `remark-rehype` → `rehype-raw` → **sanitise** → `KaTeX` → `Shiki` → `Mermaid placeholders` → `asset and link rewriting` → `heading ids and index tree` → `table wrapping` → `drop-cap marking` → `stringify`. Word count and reading time (220 wpm) are computed.
4. Services called: none. Mermaid is drawn later in the browser and loaded lazily.
5. Data stored: the `RenderResult` is cached in memory per set. Neighbouring documents are prefetched when the browser is idle.
6. Asynchronous work: Shiki grammar loading, Mermaid drawing, and idle prefetch.
7. Result: HTML in the paper reader, a heading index, and a repair list (line, before, after) if any repairs were made.
8. What can fail: a render error shows a failed state for that document only. A link to a markdown file that is not in the set is marked missing. Image lookup falls back to a basename match and refuses ambiguous matches. An unresolved image shows a placeholder (README and `tasks/todo.md` describe this; the component itself was not read for this document).

4.1.3 Workflow: Export self-contained HTML

1. Trigger: the user picks HTML export in the export bar (`src/reader/ExportBar.tsx`; not read, so the exact UI is uncertain — verify with developer).
2. Received by: `buildStandalone` in `src/export/standalone.ts`, entirely in the browser.
3. Processing: every document is placed in an off-screen host, so unopened documents are included. Diagrams already drawn on screen are copied, and the rest are drawn in the off-screen host (Mermaid needs layout to measure text). Images are fetched from their object URLs and converted to data URIs. All stylesheets are read from `document.styleSheets` with `url()` references inlined.
4. Services called: none.
5. Data stored: none. The file is downloaded via a temporary object URL.
6. Asynchronous work: fetching and encoding assets, diagram drawing.
7. Result: one `.html` file that works with the network disabled. The filename is sanitised by `safeFilename`.
8. What can fail: an image that cannot be read loses its `src`. A cross-origin stylesheet is skipped, and none are expected. Diagrams must be settled before serialising, because the print context runs without JavaScript.

4.1.4 Workflow: Export PDF

1. Trigger: the user picks PDF export.
2. Received by: `requestPdf` (`src/export/pdf.ts`) builds on the HTML export. It rejects locally if the HTML is over 20 MB, then POSTs `{ html, title, format }` to `/api/export/pdf`.
3. Processing (server): the body is validated, and format is Letter or otherwise
4. `renderPdf` rejects with 429 if the queue is saturated. Otherwise it waits for

a slot (concurrency is CPU count, clamped to 1-4). It creates a fresh browser context with JavaScript off and service workers blocked, and denies every request except data:, blob: and about:. It loads the HTML, waits for fonts, emulates print media and prints with a running title header and page / total footer. It closes the context in a finally.

4. Services called: the local Chromium instance only.
5. Data stored: none. The HTML is in memory for one request.
6. Asynchronous work: a queue, a 20 s setContent timeout, a 20 s context default timeout and a 30 s overall deadline.
7. Result: application/pdf as an attachment. The client downloads it.
8. What can fail: 400 for a missing or empty html. 413 if the body is over 20 MB. 429 when busy. 504 on deadline. 503 for any other failure, including Chromium being unavailable. See API.md and TROUBLESHOOTING.md.

4.1.5 Workflow: Health check and shutdown

1. Trigger: the Docker HEALTHCHECK every 30 s, or an operator calling GET /healthz.
2. Received by: server/index.ts.
3. Processing: returns status, browser (up or idle), the queue {active, pending} and uptime. It does not launch a browser. browser: idle is normal before the first PDF request.
4. On SIGTERM or SIGINT the server closes, then closes the browser, then exits 0.
5. What can fail: if the process does not answer, the container is marked unhealthy after 3 retries. The Dockerfile does not restart it by itself. Compose's restart: unless-stopped acts on exit, not on unhealthy status.

Chapter 5

Data Model

Entities, relationships, constraints, ownership and business rules.

5.1 Overview

There is no database. The data model is a set of in-memory TypeScript types in `src/types/domain.ts`, created when a folder is dropped and discarded on reset or reload. `specification.json` is said to hold the matching schemas but was not read for this document: uncertain — verify with developer.

5.2 Entities

Entity	Purpose	Key fields
DocumentSet	One drop.	<code>id</code> , <code>rootName</code> (folder name or DocumentSet name)
MarkdownDoc	One markdown file.	<code>id</code> (FNV-1a hash of the normalised path), <code>path</code> (lowercased normalised key), <code>content</code>
AssetRef	A non-markdown image.	<code>path</code> , <code>reason</code>
SkippedFile	A file not used.	<code>path</code> , <code>reason</code>
RenderResult	Output of the pipeline for one document.	<code>docId</code> , <code>html</code> (sanitised), <code>headings</code> (tree)
Heading	Index tree node.	<code>id</code> (slug), <code>depth</code> 1-6, <code>text</code> , <code>children</code>
RepairNote	One recorded repair.	<code>rule</code> , <code>line</code> , <code>before</code> , <code>after</code> , <code>detail</code>
MermaidBlock	A diagram to draw.	<code>id</code> (content-hashed), <code>code</code>
IngestError	Failure of the whole drop.	<code>code</code> : <code>too-large</code> , <code>too-many-files</code> , <code>not-found</code>

`RepairRule` is one of `heading-skip`, `duplicate-h1`, `list-nesting`, `mixed-markers`, `unbalanced-emphasis`.

5.3 Relationships

A `DocumentSet` has many `MarkdownDoc`, many `AssetRef` and many `SkippedFile`. Each `MarkdownDoc` has zero or one `RenderResult` in the render cache (a `WeakMap` keyed by the set object), which has many `Heading`, `RepairNote` and `MermaidBlock`. Documents refer to each other and to assets by relative path, resolved by `src/ingest/links.ts` and `assets.ts`.

5.4 Constraints and Indexes

- Limits (LIMITS): 2000 files, 200 MB total, 10 MB per markdown file, 20 MB PDF body, 30 s render deadline.
- Recognised markdown extensions: `.md`, `.markdown`, `.mdown`, `.mkd`. Image extensions: `.png` `.jpg` `.jpeg` `.gif` `.webp` `.avif` `.svg` `.bmp` `.ico`.
- `MarkdownDoc.id` derives from the path only. Two documents cannot share a path. Hash collisions are not handled: uncertain — verify with developer.
- Asset keys are normalised (backslashes, `./`, duplicate slashes, leading slash, lowercase). On a duplicate key the first file wins.
- The link index maps paths, extensionless paths and directory paths (to README or index). A key that matches two different documents is removed as ambiguous.
- `DocumentSet.id` includes `Date.now()`, so it is not stable across drops.
- Reading order is by `sortKey` (see below). Ties break on path.

5.5 Ownership and Retention

The set is owned by the browser tab that created it (the Zustand store). It has no retention: reset or replacing the set revokes all object URLs, and a reload discards everything. The server retains nothing. The render cache is weakly keyed to the set. A failed document is retained in the set with error set.

5.6 Migrations

Not applicable: no persisted schema.

5.7 Business Rules

- **Title:** `frontmatter.title`, else first ATX or setext H1, else prettified filename (numeric prefix stripped, words capitalised).
- **Reading order,** highest priority first: directory grouping (root files first, nested directories together); explicit frontmatter order (numeric or numeric string); index names (`readme`, `index`, `introduction`, `intro`, `overview`, `start`, `getting-started`, `home`, `00`, `_index`, `summary`); numeric filename prefix (`01-`, `2_`, `003.`); natural alphabetical. Index rank deliberately outranks numeric prefix. Missing values sort last within their rank.
- **Ignored directories:** `.git`, `.svn`, `.hg`, `node_modules`, `.next`, `.nuxt`, `dist`, `build`, `.cache`, `.idea`, `.vscode`, `__pycache__`, `.venv`, `venv`. Also `.DS_Store` and `Thumbs.db`.
- **Frontmatter:** a leading `---` block (an optional BOM is allowed). Invalid YAML is left in the body as content.
- **Common root:** if every file shares one top-level folder, it becomes `rootName` and is stripped from paths.
- **Repairs** never touch the source, are always reported, and:
 - drop a leading H1 that repeats the document title;
 - demote later H1s to H2 and shift their subtree;
 - promote a heading with no parent;
 - pull a heading with a skipped level up to one below its parent;

- lift the children of a list item that contains only a list;
- normalise the tight/loose spacing of a list;
- report unclosed `**` or `__` as rendered literally.
- **Reading time:** words ÷ 220, rounded, minimum 1.
- **Links:** classified internal, external, anchor or missing. A relative `.md` link that resolves to nothing is missing.
- **Images:** a lookup falls back to a basename match, and refuses ambiguous ones.

Chapter 6

API

Behaviour, assumptions and side effects that a generated API spec does not capture.

6.1 Overview

The server (`server/index.ts`) exposes two application endpoints plus static hosting. There is no authentication, versioning or CORS configuration in the code (no CORS plugin is registered). No generated API spec exists in the repository. All errors are `JSON { "error": string }`, except the SPA fallback. General conventions are in `DESIGN.md`.

6.2 Endpoints

6.2.1 GET /healthz

- Purpose: liveness and light status. Used by the Docker HEALTHCHECK.
- Authentication: none.
- Parameters: none.
- Response example:

```
{ "status": "ok", "browser": "idle", "queue": { "active": 0, "pending": 0 }, "up": true }
```
- browser is up only if a Chromium instance has been launched and is connected. idle is normal before the first PDF request and after a browser disconnect. The call never launches a browser.
- Side effects: none.

6.2.2 POST /api/export/pdf

- Purpose: print already-rendered, self-contained HTML to PDF.
- Authentication: none.
- Request: `application/json`, body limit 20 MB.

```
{ "html": "<!doctype html>...", "title": "Handbook", "format": "A4", "landscape": true }
```

 - `html` (required): non-empty string. Must inline all assets, because every request except `data:`, `blob:` and `about:` is aborted and JavaScript is disabled. Remote images and stylesheets will not load.
 - `title`: string, default Documents. Printed in the page header (escaped, first 120 characters) and used for the filename.

- format: Letter or anything else falls back to A4.
- landscape: honoured only when exactly true. The bundled client (src/export/pdf.ts) does not send it.
- Response 200: application/pdf, content-disposition: attachment; filename="<ascii-title>.pdf". The filename is reduced to [\w.-], at most 80 characters, and falls back to documents.
- Output: margins 22/18/20/22 mm (top/right/bottom/left), preferCSSPage-Size: true, background printing on, print media, en-GB locale, UTC timezone, header with title, footer page / total.
- Dependencies: the local Chromium via playwright-core.
- Side effects: launches a shared Chromium on first use. Creates and always closes one browser context per request. Nothing is written to disk by application code. The HTML is not logged by application code.
- Concurrency: at most clamp(CPU count, 1, 4) renders run at once. More than 4× that many pending requests get 429.

6.2.3 Static files and fallback

- GET / and other paths serve STATIC_ROOT (the Vite build). index.html is no-cache. Files whose name contains a hash of eight or more hex characters get public, max-age=31536000, immutable.
- Any other unmatched path returns index.html (SPA fallback), except paths starting /api/, which return JSON 404 { "error": "Not found" }.
- Every response gets x-content-type-options: nosniff, referrer-policy: no-referrer, cross-origin-opener-policy: same-origin.

6.3 Errors

Status	Endpoint	Cause	Body / header
400	POST /api/export/pdf	Missing body, html not a string, or blank	Expected
413	POST /api/export/pdf	Body over 20 MB	Fastify d
429	POST /api/export/pdf	Queue saturated	The prin
504	POST /api/export/pdf	30 s render deadline exceeded	Advises
503	POST /api/export/pdf	Any other failure, including browser launch failure	The prin
404	/api/*	Unknown API path	{ "error

Malformed non-JSON content types were verified to produce 400 according to tasks/todo.md. The exact framework behaviour was not confirmed in code: uncertain — verify with developer. See TROUBLESHOOTING.md for diagnosis.

Chapter 7

Security

Security architecture, trust boundaries and who is responsible for what.

7.1 Trust Boundaries

1. **Dropped files → browser pipeline.** Markdown and any embedded HTML are untrusted. The boundary is `rehype-sanitize` in `src/pipeline/render.ts`, placed directly after `rehype-raw`. Everything after it is generated by the application from a clean tree. See ADR-002.
2. **Browser → POST /api/export/pdf.** The endpoint accepts whatever a caller posts, not only output from this client. It is treated as hostile HTML rendered in a real browser. ADR-003
3. **Network → container.** The service has no authentication, so exposure is controlled by port binding.

Content leaves the browser only at PDF export. It is held in memory for one request and never written to disk. The HTML export is local.

7.2 Authentication

None, by design. The README and `docker-compose.yml` state that the app has nothing to authenticate. Anyone who can reach port 8080 can use the print service.

7.3 Authorisation

None. There are no users, roles or protected resources.

7.4 Secrets Management

No secrets exist in the repository, image or Compose file. Nothing is read from the environment except `PORT`, `HOST`, `STATIC_ROOT`, `LOG_LEVEL` and `NODE_ENV`.

7.5 Session Handling

No sessions or cookies are set by the server. Each print request uses a fresh browser context, so no cookies, storage or service workers carry between documents.

7.6 API Security

Controls on the print endpoint:

Control

JavaScriptEnabled: false

Context-level default-deny routing (only data:, blob:, about: continue)

Fresh BrowserContext per request, closed in finally; service workers blocked

p-limit (1-4) plus queue cap; 429 when saturated

bodyLimit 20 MB; 20 s setContent, 20 s default and 30 s overall deadlines

title HTML-escaped in the header template and truncated to 120 characters; download filename

Generic 503 body, detail logged only

Response headers x-content-type-options, referrer-policy and cross-origin-opener-policy are set on every response. No Content-Security-Policy header is set by the server: uncertain — verify with developer whether one is intended.

The README and tasks/todo.md report these controls as verified by probe (tests/e2e/adversarial.mjs); this document has not re-run them.

7.7 Network Exposure

Compose publishes 127.0.0.1:8080:8080 only. Binding to 0.0.0.0 would publish a headless browser that renders arbitrary posted HTML. The server itself sets trust-Proxy: false. The container runs as non-root pwuser (so the Chromium sandbox stays enabled), with a read-only root filesystem, all capabilities dropped and no-new-privileges. ipc: host is set because Chromium crashes without it, which slightly weakens isolation.

7.8 Input Validation

- **Markdown/HTML:** the sanitiser schema strips script, style, iframe, object, embed, link, meta, base and form. Link href schemes are limited to http, https, mailto, tel, irc, ircs, xmpp. Image src is an explicit allowlist: http(s), blob:, raster data:image/* (SVG excluded) and relative paths. Task-list input is pinned to disabled checkboxes. id and name are clobbered with the user-content- prefix by the sanitiser. Headings get unprefixed ids afterwards, downstream.
- **Server body:** html must be a non-empty string. format accepts Letter, otherwise A4. title must be a string. landscape is honoured only when exactly true.
- **Ingest limits:** 2000 files, 200 MB total, 10 MB per markdown file.
- **KaTeX** renders errors inline and does not throw.

7.9 Dependency Security

package-lock.json is committed and installs use `npm ci`. No automated vulnerability scanning or update bot is configured in the repository. playwright-core is pinned to the exact version of the base image. The rendering libraries define the sanitisation surface, so upgrade them with the pipeline tests and adversarial probe. See DEPENDENCIES.md.

7.10 Logging and Audit

The server logs through Fastify's logger. Failures are logged at error. There is no audit log, and no request bodies are logged by application code. Fastify's default request logging may record URLs: uncertain — verify with developer.

7.11 Sensitive Data

User documents are the only sensitive data. They stay in the browser except for PDF export, when they are held in memory for one request. They may be disclosed only via the PDF response to the same caller. Nothing is persisted, and /tmp is the only writable path. Whether Chromium writes document content to /tmp during rendering: uncertain — verify with developer.

7.12 Rate Limiting

No per-client rate limit. The service uses a global concurrency limit and a queue cap (429 beyond $4 \times$ concurrency pending). One client can consume all capacity.

7.13 Encryption and TLS

The server speaks plain HTTP. TLS termination is not provided in the repository, so a reverse proxy is required for any non-local deployment. No data is encrypted at rest, because none is stored.

7.14 Security Assumptions

- The service runs on loopback or a trusted network.
- The client inlines all assets before posting, so blocking all network access in the print browser costs nothing.
- The print browser has JavaScript disabled, so the HTML has no need to execute.
- src/pipeline/sanitize.ts exports a printSchema described as a server-side second filter. **It is not used anywhere**, and the server does not sanitise posted HTML. The controls above are the only protection.
- Mermaid is described as running in `securityLevel: 'strict'` (tasks/todo.md). The component was not read to confirm this.

7.15 Responsibilities

- **Developers:** keep the sanitiser before the generators and add tests for hostile payloads. Keep the print-service controls layered. Re-run `npm run test:security` after changes.
- **Operators:** keep port 8080 off untrusted networks, add TLS and access control in front of it if it is exposed, and keep resource limits.
- **Users:** decide whether to use PDF export for sensitive documents. The HTML export stays local.

Chapter 8

Deployment

How the system is built, configured and deployed, and how to roll back.

8.1 Environments

- **Local development:** Vite dev server (`npm run dev`) for the client and `npm run dev:server` (`tsx watch server/index.ts`) for the server.
- **Local container:** `docker compose up --build`, serving on `http://localhost:8080`, bound to `127.0.0.1`. The image is tagged `markdown-visualiser:local`.
- **Staging / production:** none are defined in the repository. Where and how it is hosted: uncertain — verify with developer.
- **CI:** the only workflow is `docforge.yml` (documentation checks and manual build). There is no build, test or deploy pipeline for the application: uncertain — verify with developer.

The dev-server proxy from Vite to the print service was not checked (`vite.config.ts` was not read), so PDF export in `npm run dev` may need the container: uncertain — verify with developer.

8.2 Build

`npm run build` runs `vite build` (client into `dist/`) and then `esbuild` to bundle `server/index.ts` into `dist-server/index.js`. It targets Node 20, ESM, and keeps packages external, so `node_modules` must exist at runtime.

The Dockerfile has two stages:

1. `build (node:24-bookworm-slim)`: `npm ci`, then `npm run typecheck && npm run build`. A type error fails the image build.
2. `runtime (mcr.microsoft.com/playwright:v1.62.1-noble)`: `npm ci --omit=dev`, then copies `dist/` and `dist-server/` and switches to user `pwuser`.

`playwright-core` is pinned to exactly 1.62.1 in `package.json` to match the image tag. The pin and the tag must move together.

`init.sh` runs the full local bootstrap: it checks Node 20 or later, runs `npm ci`, vendors design skills, typechecks, builds and runs tests.

The runtime image is about 3.63 GB, because the base ships three browsers and only Chromium is used. Slimming it is a recorded but unstarted follow-up (tasks/todo.md).

8.3 Deploy

Local and single-host:

```
docker compose up --build -d
curl -sf localhost:8080/healthz
```

Compose builds the runtime target and runs it with `read_only: true`, a 512 MB `/tmp` tmpfs, `ipc: host`, `init: true`, `cap_drop: ALL`, `no-new-privileges`, `mem_limit: 2g`, `pids_limit: 512` and `restart: unless-stopped`.

Do not publish port 8080 beyond loopback without putting access control in front of it. The service has no authentication and renders posted HTML in a headless browser (see SECURITY.md).

No registry push, orchestration manifests or hosted deployment steps exist in the repository.

8.4 Configuration

Variable	Default	Set by	Meaning
PORT	8080	Dockerfile	Listen port
HOST	0.0.0.0	Dockerfile	Listen address inside the container
STATIC_ROOT	/app/dist in the image	Dockerfile	Directory served as the SPA
LOG_LEVEL	info	Compose	Fastify/Pino log level
NODE_ENV	production	Dockerfile, Compose	Standard

There are no secrets and no config files. Limits (upload caps, body limit, timeouts, concurrency) are code constants. See DESIGN.md.

The HEALTHCHECK fetches `http://127.0.0.1:8080/healthz` every 30 s (20 s start period, 3 retries), so it assumes the default port. Changing PORT without changing the health check makes the container report unhealthy.

8.5 Rollback

The application is stateless, so rollback means running the previous image. Nothing needs undoing in a database or volume.

Because the only image tag defined is `markdown-visualiser:local`, a rebuild overwrites it. To keep a rollback target, tag the working image (for example `docker tag markdown-visualiser:local markdown-visualiser:<version>`) before rebuilding. Alternatively check out the previous commit and run `docker compose up --build -d`. No release tagging or registry process is defined: uncertain — verify with developer.

Chapter 9

Testing

Testing strategy and what must pass before a change is accepted.

9.1 Strategy

Three layers. Unit tests (Vitest, jsdom) cover ingest, pipeline and tokens. Browser drives run the real served bundle in Chromium. An adversarial probe attacks the print endpoint. The project's history shows the unit suite stayed green while several real bugs existed. Those were found only by driving Chromium or inspecting screenshots (tasks/todo.md), so do not treat green unit tests as sufficient for UI, export or server changes.

definition_of_done.md lists criteria with the command that proves each one.

9.2 Unit Tests

- Runner: Vitest 3, environment: 'jsdom', tests/setup.ts as setup, @ aliased to src. Includes tests/**/*test.ts(x).
- Files: tests/unit/docset.test.ts, fixtures.test.ts, ordering.test.ts, pipeline.test.ts, tokens.test.ts, walker.test.ts.
- Run: npm test (watch mode) or npm test -- --run (once).
- Count: the README and tasks/todo.md report 121 tests at the last recorded gate. This was not re-run.
- The definition of done refers to filters such as npm test -- links or -- sanitize. Those match test file names, and not all such files exist. For example, there is no links.test.ts or sanitize.test.ts among the listed files, so those may live inside pipeline.test.ts or docset.test.ts: uncertain — verify with developer.

9.3 Integration Tests

None separate from the unit and browser suites. tests/fixtures/handbook/ is a fixture folder (nested directories, an image, a messy markdown file, a non-markdown file) used by unit tests and the browser drives.

9.4 End-to-End Tests

`npm run test:e2e` (`tests/e2e/run.sh`) regenerates fixtures (`make-fixtures.mjs`) and runs each drive in a sibling container that shares the app container's network namespace, using the built image.

Drive	Covers
<code>drive.mjs</code> (ingest)	walker, reading order, grouping, titles, skips
<code>drive-reader.mjs</code> (reader)	navigation, keyboard, responsiveness, axe accessibility, reduced motion
<code>drive-export.mjs</code> (export)	both exports, re-opened with the network blocked
<code>shots.mjs</code>	screenshots into <code>tests/e2e/shots</code> (run as part of all)

Run one with `bash tests/e2e/run.sh reader` (names: `ingest`, `reader`, `export`, `adversarial`). Requires the app container running: default name `markdown_visualiser-app-1`, image `markdown-visualiser:local`.

9.5 Security Tests

`npm run test:security` runs `tests/e2e/adversarial.mjs` against the running container. It covers `file://` reads, internal and metadata addresses, CSS-only SSRF, script execution, oversized bodies (413), concurrency saturation (429), malformed bodies (400) and error-text disclosure. The last recorded result is 11 of 11 checks (`tasks/todo.md`). This was not re-run.

Sanitiser behaviour is covered by hostile-payload unit tests: `tasks/todo.md` says 13 payload tests exist. The unit tests were not read for this document.

9.6 Manual and Smoke Tests

```
docker compose up --build -d
curl -sf localhost:8080/healthz           # status ok, browser idle before first PDF
```

Then open `http://localhost:8080`, drop `tests/fixtures/handbook`, click through the index, export HTML and PDF, and open the HTML with the network disabled. Definition-of-done items 4.1 and 4.2 are manual checks.

9.7 Test Environments and Data

Unit tests need only Node (engines says 20 or later; the image builds with 24). E2E and security tests need Docker. Fixtures live in `tests/fixtures/handbook/` and are also regenerated by `tests/e2e/make-fixtures.mjs`. No external services, credentials or databases are involved.

9.8 Running Tests

```
npm run typecheck
npm run lint           # --max-warnings 0
npm test -- --run
```

```
docker compose up --build -d
npm run test:e2e
npm run test:security
```

9.9 Required Before Changes

- Every change: `npm run typecheck`, `npm run lint`, `npm test -- --run` must pass with no warnings.
- Changes to `src/pipeline/`, `src/export/`, `src/reader/` or `src/ingest/`: also `npm run test:e2e`.
- Changes to `server/`, `Dockerfile`, `docker-compose.yml` or `sanitize.ts`: also `npm run test:security`.
- No CI job runs these checks. The only workflow is the docforge documentation check, so they must be run locally: uncertain — verify with developer.

Chapter 10

Operations

Running the system day to day: monitoring, alerts, routine tasks, incidents.

10.1 Monitoring

- **GET /healthz** returns status, browser (up or idle), queue (active, pending) and uptime in seconds. idle is normal until the first PDF request.
- **Docker health check:** runs /healthz every 30 s with a 5 s timeout, a 20 s start period and 3 retries. Check with `docker ps` or `docker inspect`.
- **Logs:** `docker compose logs -f app` (JSON, level from LOG_LEVEL).
- **Resources:** the container is limited to 2 GB memory and 512 pids. Watch `docker stats`. Chromium is the main consumer.
- No metrics endpoint, dashboards or tracing exist in the repository.

10.2 Alerts

None are configured in the repository. Suggested signals (not implemented): the container going unhealthy, repeated pdf render failed log lines, sustained queue.pending above 0, and 429, 503 or 504 responses. Alerting infrastructure: uncertain — verify with developer.

10.3 Routine Operations

- **Deploy or update:** `docker compose up --build -d` (see DEPLOYMENT.md).
- **Restart:** `docker compose restart app`. Chromium is relaunched on the next PDF request.
- **Smoke check after any change:** `curl -sf localhost:8080/healthz`, then export a PDF from the UI.
- **Regression check:** `npm run test:e2e` and `npm run test:security`. Both need the container running.
- No scheduled jobs, backups, log rotation or data clean-up exist. The service is stateless.
- Docker's default logging behaviour applies to logs, so long-running hosts may need log rotation configured outside the repository: uncertain — verify with developer.

10.4 Incident Response

1. **Is it up?** `curl localhost:8080/healthz`. If it fails, run `docker compose ps` and `docker compose logs --tail 200 app`. The server exits 1 if it cannot listen.
2. **Only PDF export failing?** The web UI and HTML export do not depend on the print service, so users can continue with HTML export. See the 429, 503 and 504 entries in TROUBLESHOOTING.md.
3. **Overload or memory pressure:** restart the container. Check that `ipc: host`, `init: true` and the memory limit are still in Compose.
4. **Suspected abuse of the endpoint:** the service is unauthenticated, so first confirm it is bound to 127.0.0.1 (`docker compose ps`, port mapping) and not published wider. If it was exposed, restrict access at the network level before investigating, and see SECURITY.md.
5. **Bad release:** roll back per DEPLOYMENT.md.

Escalation contacts and on-call arrangements: uncertain — verify with developer.

Chapter 11

Maintenance

Written for the engineer who inherits this system.

11.1 Starting and Stopping

- Start: `docker compose up --build -d`. Stop: `docker compose down`.
- The server handles `SIGTERM` and `SIGINT` by closing Fastify, then closing Chromium, then exiting 0. `docker stop` therefore shuts down cleanly.
- Chromium is launched lazily on the first PDF request, not at startup. `GET /healthz` reports browser: idle until then and after a browser crash. This is not a fault.
- Compose sets restart: unless-stopped.

11.2 Dependency Updates

- `package-lock.json` is committed, and the Docker build uses `npm ci`.
- Runtime dependencies are only `fastify`, `@fastify/static`, `p-limit` and `playwright-core`. Everything else is a build-time dependency, because Vite bundles the client. Keep new client libraries in `devDependencies`, or the runtime layer grows (comment in `package.json`).
- **playwright-core and the base image tag must be updated together** (1.62.1 in both `package.json` and `Dockerfile`). A mismatch breaks the browser executable lookup (comment in `Dockerfile`).
- The unified/remark/rehype packages, `hast-util-sanitize`, `Shiki`, `KaTeX` and `Mermaid` define rendering and the trust boundary. After upgrading, run the unit tests and the adversarial probe (see `TESTING.md`).
- `eslint` runs with `--max-warnings 0`, so new lint warnings after an upgrade fail `npm run lint`.
- Dependency audit tooling and update cadence (Dependabot or similar): none found in the repository.

11.3 Database Migrations

Not applicable. There is no database and the application persists nothing. See `DATA_MODEL.md`.

11.4 Secret Rotation

Not applicable. No secrets, credentials or API keys were found in the repository, image or Compose file. If the docforge CI install (DOCFORGE_SPEC) requires credentials: uncertain — verify with developer.

11.5 Configuration Management

Configuration is a handful of environment variables (see DEPLOYMENT.md). Hard limits live in code. Two values are duplicated and must be edited together: the 20 MB PDF body limit is in LIMITS.pdfBodyLimitBytes (src/types/domain.ts) and in PDF_BODY_LIMIT (server/index.ts). The 30 s render deadline appears as LIMITS.pdfRenderDeadlineMs and as RENDER_DEADLINE_MS in server/pdf.ts.

11.6 Logs

The server writes structured JSON logs (Fastify/Pino) to stdout, and the level comes from LOG_LEVEL. Read them with `docker compose logs -f app`. PDF failures are logged at error with the underlying error. The HTTP response to the client is intentionally generic. No log shipping, retention or aggregation is configured in the repository. See OPERATIONS.md.

11.7 Backups

Nothing to back up. The service is stateless and stores no user documents (the container root is read-only, with only /tmp writable). The source repository is the only asset.

11.8 Releases

package.json says version 1.0.0 and the project is private. No release process, tagging convention or automated release pipeline is defined. The docforge workflow builds a PDF manual on manual dispatch or on tags (refs/tags/). Everything else about releasing: uncertain — verify with developer. Record changes in CHANGELOG.md under [Unreleased].

11.9 Making Production Changes

1. Run `npm run typecheck`, `npm run lint` and `npm test`. The Docker build also runs the typecheck.
2. Build the image and start it: `docker compose up --build -d`.
3. Run `npm run test:e2e`, which needs the container up. Its default container name is `markdown_visualiser-app-1`, overridable with `CONTAINER`, and `IMAGE` similarly. Run `npm run test:security` for any change under `server/` or `src/pipeline/`.
4. Meet the rest of the criteria in `definition_of_done.md` where relevant.
5. Update the affected docs. `docforge.toml` maps changed files to docs, and CI runs `docforge check --strict`.

11.10 Known Technical Debt

- **Image size.** The runtime image is 3.63 GB because it uses the full Playwright image. Proposed fix: `node:24-slim` plus `playwright install --with-deps chromium` (about 1.2 GB, an estimate from the authors). Not done.
- **Unused printSchema.** `src/pipeline/sanitize.ts` exports a stricter server-side schema, described as applied before Chromium. Nothing imports it, and the server has no sanitiser. The print service relies on JavaScript being disabled and on request blocking instead. Either wire it in or delete it.
- **Duplicated constants** between client and server (see Configuration Management).
- **Very large sets:** virtualised rendering is deferred until a real set proves slow (`tasks/todo.md`).
- `tests/e2e/run.sh` hard-codes the Compose container name and has Windows path handling. DoD lists `tests/adversarial/pdf-endpoint.sh`, which does not exist; the probe is `tests/e2e/adversarial.mjs`.
- The `docforge.toml` project name is `pilot-live`, which does not match the package name `markdown-visualiser`.

11.11 Fragile Areas

11.11.1 Fragile area: sanitiser schema and pipeline order

- Component: `src/pipeline/sanitize.ts`, `src/pipeline/render.ts`
- Why it is sensitive: the sanitiser is the only defence between dropped markdown and the DOM. A src allowance must have the inherited bare 'src' removed (`inheritedExcept`), or the stricter rule constrains nothing. This caused a real `data:text/html XSS` during development.
- What depends on it: the reader (`dangerouslySetInnerHTML`), the HTML export, and the PDF service.
- What must be tested before changing it: `npm test` (pipeline and sanitise tests) and a check that KaTeX and Shiki output still survives.
- Relevant files: `src/pipeline/sanitize.ts`, `src/pipeline/render.ts`, `src/pipeline/plugins/rehype-assets.ts`.
- Relevant ADR: ADR-002

11.11.2 Fragile area: print service

- Component: `server/pdf.ts`
- Why it is sensitive: it renders caller-supplied HTML in Chromium. `javascriptEnabled: false`, the context-level route that blocks everything except `data:`, `blob:` and `about:`, the per-request context, and the queue cap are layered controls. `waitUntil: 'load'` is deliberate, because `networkidle` would wait for the timeout on every render.
- What depends on it: PDF export.
- What must be tested before changing it: `npm run test:security` (adversarial probe) and `npm run test:e2e export drive`.
- Relevant files: `server/pdf.ts`, `docker-compose.yml`, `Dockerfile`.
- Relevant ADR: ADR-003, ADR-007

11.11.3 Fragile area: folder walking and ordering

- Component: `src/ingest/walker.ts`, `src/ingest/ordering.ts`
- Why it is sensitive: `readEntries` returns batches (about 100 in Chromium) and must be drained in a loop, or large folders silently lose files. The drop's item list must be read synchronously before the first `await`.
- What must be tested before changing it: `tests/unit/walker.test.ts`, `tests/unit/ordering.test.ts`, and the ingest drive.
- Relevant files: `src/ingest/walker.ts`, `src/ingest/ordering.ts`.
- Relevant ADR: ADR-004, ADR-006

11.11.4 Fragile area: export serialisation

- Component: `src/export/standalone.ts`
- Why it is sensitive: diagrams must be static SVG before serialising, because the print context has no JavaScript. Images must be retyped by extension (`assets.ts`), or exports get `data:text/plain` images. Both went wrong during development and were fixed (`tasks/todo.md`).
- What must be tested before changing it: the export drive (`tests/e2e/drive-export.mjs`), which re-opens both exports with the network blocked.
- Relevant files: `src/export/standalone.ts`, `src/ingest/assets.ts`, `src/styles/print.css`.

Chapter 12

Troubleshooting

Practical guide. Keep historical lessons even after the bug is fixed.

12.1 Known Problems

Entries marked (*historical*) were found and fixed during development (recorded in `tasks/todo.md`). They are kept because the same symptom can return.

12.1.1 Problem: PDF export returns 429 "The print service is busy"

- Possible symptoms: export fails with a busy message. The response has `retry-after: 5`.
- Likely causes: more than `concurrency × 4` requests pending. Concurrency is the CPU count clamped to 1–4.
- Diagnostic steps: `curl localhost:8080/healthz` and look at `queue.active` and `queue.pending`.
- Resolution: retry after a few seconds. For sustained load, more CPUs raise concurrency up to 4. The container memory limit is 2 GB.
- Relevant logs: none are written for 429s.
- Relevant components: `server/pdf.ts` (`BusyError`), `server/index.ts`.
- Escalation considerations: sustained 429s with a lightly used system suggest stuck renders. Check `docker compose logs app`.

12.1.2 Problem: PDF export returns 504 or "Rendering took too long"

- Possible symptoms: export fails after about 30 s.
- Likely causes: a very large or complex document. Fonts or layout never settling.
- Diagnostic steps: check the HTML export size. Try the HTML export.
- Resolution: use the self-contained HTML export and print from the browser (the error message says this).
- Relevant components: `server/pdf.ts` (`RENDER_DEADLINE_MS 30 s`, `SETCONTENT_TIMEOUT_MS 20 s`).
- Escalation considerations: raising deadlines needs load testing, and `LIMITS.pdfRenderDeadlineMs` in `src/types/domain.ts` must be kept in step.

12.1.3 Problem: PDF export returns 503 “The print service is unavailable”

- Possible symptoms: generic 503 with no detail.
- Likely causes: Chromium failed to launch or crashed. Possible causes are memory pressure or missing ipc: host. Base image or playwright-core version mismatch. The read-only root filesystem needing a writable location beyond /tmp. Which of these applies must be read from the log.
- Diagnostic steps: docker compose logs app and search for pdf render failed. Check /healthz. browser: idle right after a failure is expected, because the browser handle is cleared on disconnect and relaunched on the next request.
- Resolution: confirm playwright-core and the image tag are both 1.62.1. Confirm Compose still has ipc: host, init: true and the /tmp tmpfs. Restart the container.
- Relevant logs: pdf render failed at error level.
- Relevant components: server/pdf.ts, Dockerfile, docker-compose.yml.
- Escalation considerations: the client sees no detail by design. Do not expose the underlying error to callers.

12.1.4 Problem: PDF export rejected with “over the 20 MB limit” or HTTP 413

- Possible symptoms: message from the client, or a 413 from the server.
- Likely causes: many or large images inlined as data URIs.
- Resolution: use the HTML export, or reduce image sizes. The limit is in two places (src/types/domain.ts and server/index.ts).
- Relevant components: src/export/pdf.ts, server/index.ts.

12.1.5 Problem: Files missing from a large dropped folder (*historical*)

- Possible symptoms: files “randomly” absent from the index.
- Likely causes: readEntries returns batches, so calling it once truncates. Also a file is skipped once the 2000-file limit is hit, or an unreadable file is ignored by the walker.
- Diagnostic steps: check the skipped list and the count. Run tests/unit/walker.test.ts.
- Resolution: keep the drain loop in walker.ts. Split the folder if it exceeds the limit.
- Relevant components: src/ingest/walker.ts.

12.1.6 Problem: Images blank or broken in exported HTML (*historical*)

- Possible symptoms: images fine in the reader but not in the exported file. The offline check can still pass in the same browser session, because blob URLs stay valid there.
- Likely causes: files from a directory drop often have an empty MIME type, so the object URL was untyped and the data URI became data:text/plain.
- Resolution: assets are retyped from the file extension in buildAssetMap. Check that extension mapping in src/ingest/assets.ts includes the format.
- Relevant components: src/ingest/assets.ts, src/export/standalone.ts.

12.1.7 Problem: Mermaid diagrams show source, or are missing from the export (*historical*)

- Possible symptoms: diagram source text displayed instead of a diagram. Or a placeholder in the PDF or HTML export for documents that were never opened.
- Likely causes: a failed dynamic import of the Mermaid chunk was cached permanently. Or the exporter copied only diagrams already drawn on screen. The print context has JavaScript disabled, so placeholders never become diagrams.
- Resolution: the exporter now draws diagrams for every document in an off-screen host before serialising. If the problem returns, check the Mermaid loader for a cached rejected promise (`src/reader/Mermaid.tsx`, not read for this document).
- Relevant components: `src/reader/Mermaid.tsx`, `src/export/standalone.ts`.

12.1.8 Problem: Mismatched title and body after navigating (*historical*)

- Possible symptoms: for one frame, the new title sits above the previous document's text.
- Likely causes: render state updated only in an effect.
- Resolution: state is keyed by document id and mismatches resolve during render (`useRenderedDoc`). Preserve that when editing it.
- Relevant components: `src/state/render.ts`.

12.1.9 Problem: Heading hierarchy wrong after repair (*historical*)

- Possible symptoms: two titles on one page, or a demoted section's children appearing as its siblings in the index.
- Likely causes: repair depth must derive from the parent's *output* depth, and a promoted heading must count as the H1.
- Resolution: both are fixed in `remark-repair.ts` and covered by tests.
- Relevant components: `src/pipeline/plugins/remark-repair.ts`.

12.1.10 Problem: Container reports unhealthy

- Possible symptoms: `docker ps` shows unhealthy.
- Likely causes: the health check is hard-coded to port 8080 and it fails if PORT changed. The server failed to start (it exits 1 on listen failure).
- Diagnostic steps: `docker compose logs app`. `curl localhost:8080/healthz`.
- Relevant components: `Dockerfile`, `server/index.ts`.

12.1.11 Problem: Browser e2e drives cannot reach the app

- Possible symptoms: `npm run test:e2e` fails at start.
- Likely causes: the container is not running, or its name is not `markdown_visualiser-app-1`, which is the script default. The drives use `127.0.0.1` on purpose, since Chromium upgrades other plain-HTTP hostnames to HTTPS.
- Resolution: start the container and set `CONTAINER` and `IMAGE` if the names differ.
- Relevant components: `tests/e2e/run.sh`.

Chapter 13

Dependencies

Dependencies that would be hard to replace or could significantly affect the application.

13.1 Critical Dependencies

13.1.1 Playwright (playwright-core and the `mcr.microsoft.com/playwright` image)

- Purpose: headless Chromium for PDF generation (`server/pdf.ts`).
- Current version: `playwright-core` 1.62.1 (exact pin); base image `v1.62.1-noble`.
- Why it is used: the PDF is printed from the same HTML the reader shows, so a real browser engine is required. The image supplies Chromium and its OS dependencies but not the npm package, so `playwright-core` is installed separately.
- Replacement difficulty: medium. Print options (header and footer templates, `preferCSSPageSize`, request routing) are Playwright-specific, and the security model depends on its context-level routing.
- Upgrade considerations: change the npm pin and the Dockerfile tag together, or the executable lookup breaks. Re-run the export drive and adversarial probe.
- Known limitations: the image is 3.63 GB because it ships three browsers. `page.pdf()` takes no timeout of its own, so it is bounded through the page default timeout.

13.1.2 unified / remark / rehype ecosystem

- Purpose: the markdown-to-HTML pipeline (`unified`, `remark-parse`, `remark-gfm`, `remark-math`, `remark-frontmatter`, `remark-github-blockquote-alert`, `remark-rehype`, `rehype-raw`, `rehype-sanitize`, `rehype-katex`, `rehype-stringify`, plus `hast-util-sanitize`, `unist-util-visit`).
- Current version: `unified` ^11.0.5, `remark-rehype` ^11.1.2, `rehype-sanitize` ^6.0.0 (from `package.json` ranges).
- Why it is used: a single AST-based chain lets sanitisation be placed at an exact point.

- Replacement difficulty: high. The repair plugin, sanitise schema and asset and link plugins are written against mdast and hast.
- Upgrade considerations: the sanitiser's semantics matter. `hast-util-sanitize` allows an attribute if any definition matches, which the schema has to work around (see `MAINTENANCE.md`). Re-run the hostile-payload tests.
- Known limitations: `rehype-katex` is used without `throwOnError` and renders errors inline.

13.1.3 Fastify (`fastify`, `@fastify/static`)

- Purpose: HTTP server, static hosting, body limits, logging.
- Current version: `fastify ^5.6.1, @fastify/static ^8.2.0`.
- Replacement difficulty: low, since the server is one small file.
- Upgrade considerations: 413 for oversized bodies and the Pino logger rely on Fastify defaults.

13.1.4 p-limit

- Purpose: concurrency limit on renders.
- Current version: `^7.1.0`.
- Replacement difficulty: low. The 429 behaviour depends on its `activeCount`, `pendingCount` and concurrency.

13.1.5 Shiki

- Purpose: syntax highlighting (loaded lazily by language).
- Current version: `^4.4.3`.
- Replacement difficulty: medium. `tasks/todo.md` and `definition_of_done.md` state the JS regex engine is used so no WASM is bundled. The plugin was not read: uncertain — verify with developer.
- Upgrade considerations: highlighting output uses `--shiki-*` CSS variables that the stylesheets map onto colour (`tasks/todo.md`).

13.1.6 Mermaid

- Purpose: diagrams, drawn client-side and lazily loaded (a chunk of about 3 MB according to `tasks/todo.md`).
- Current version: `^11.17.2`.
- Replacement difficulty: medium. Diagrams must be drawn as static SVG before export, because the print context has no JavaScript.
- Upgrade considerations: it measures text, so it needs a laid-out DOM (`src/export/standalone.ts`).

13.1.7 KaTeX

- Purpose: maths rendering. Version `^0.16.22`. Its stylesheet is required to hide the MathML copy, otherwise every equation prints twice (`tasks/todo.md`).

13.1.8 React 19, Vite 7, Zustand, TypeScript

- Purpose: UI, build and client state. Versions: React ^19.2.0, Vite ^7.1.9, Zustand ^5.0.8, TypeScript ^5.9.3.
- Replacement difficulty: high for React (whole reader UI), low for Zustand (one store).
- All are devDependencies, because Vite bundles them into dist/.

13.1.9 Fonts (Fraunces, Newsreader, IBM Plex Mono via @fontsource)

- Purpose: typography, self-hosted (SIL OFL).
- Why it matters: self-hosting is what keeps the app free of third-party requests and lets the HTML export work offline.

13.1.10 Design source: MengTo/Skills

- Purpose: design direction, credited in CREDITS.md. `init.sh` vendors selected skills from the GitHub repository into `.claude/skills/mengto/`. This is not a runtime dependency, and a failed clone does not stop `init.sh`.

Chapter 14

ADR-001: Single client-side markdown pipeline

14.1 Status

Accepted

14.2 Context

The app renders markdown and exports PDF. PDF needs a real browser engine, which lives on the server.

14.3 Options Considered

- Render markdown in the browser and in a server-side pipeline for PDF.
- Render markdown only on the server.
- Render once in the browser and post the finished HTML to a print-only server.

14.4 Decision

Render only in the browser. The server receives rendered, fully inlined HTML and prints it.

14.5 Rationale

From README.md (previous version): a second server-side pipeline would drift from the client one and the PDF would stop matching the reader. Posting rendered HTML makes the PDF identical to the screen by construction and keeps the server small, stateless and free of markdown dependencies. `src/export/pdf.ts` states the server "is a print service, not a second renderer".

14.6 Consequences

- Document content leaves the browser at PDF export (in memory for one request only). HTML export stays local.
- The export must inline everything (CSS, fonts, images, diagrams) before posting.
- The server has four runtime dependencies (`package.json`).

14.7 Related Components

`src/pipeline/render.ts`, `src/export/pdf.ts`, `src/export/standalone.ts`,
`server/index.ts`, `ARCHITECTURE.md`

Chapter 15

ADR-002: Sanitiser placed immediately after rehype-raw

15.1 Status

Accepted

15.2 Context

Dropped markdown may embed raw HTML. rehype-raw turns that into real elements, which is the dangerous step. KaTeX, Shiki and Mermaid generate markup (style=, <svg>, <math>, many classes) that a sanitiser would normally reject.

15.3 Options Considered

- Sanitise last, after all generators, and whitelist their output.
- Sanitise immediately after rehype-raw, and run generators downstream.

15.4 Decision

rehype-sanitize runs immediately after rehype-raw. Generators run after it. Nothing user-controlled may be added below the boundary.

15.5 Rationale

Per comments in src/pipeline/render.ts and src/pipeline/sanitize.ts: sanitising last would force permitting style=, <svg>, <math>, <annotation> and dozens of KaTeX classes, a large surface that is what an attacker wants. The schema instead permits only marker classes the generators look for. img src is an explicit value allowlist (raster data: only, SVG excluded), because hast-util-sanitize accepts an attribute if any definition matches. A data:text/html XSS was found and fixed during development (tasks/todo.md).

15.6 Consequences

- The schema stays small. Every plugin added after the boundary must be trusted and must not pass user data through.
- rehype-assets was hardened the same way, so a downstream plugin cannot reintroduce what the sanitiser rejected (tasks/todo.md).
- Sanitiser upgrades need the hostile-payload tests.

15.7 Related Components

src/pipeline/sanitize.ts, src/pipeline/render.ts, SECURITY.md

Chapter 16

ADR-003: Contained, unauthenticated print service

16.1 Status

Accepted

16.2 Context

POST /api/export/pdf renders caller-supplied HTML in a real browser. That is a known SSRF and resource-exhaustion class, and the only meaningful attack surface. The service holds no data.

16.3 Options Considered

- Trust the client's sanitisation and print directly.
- Re-sanitise HTML on the server (a `printSchema` exists in `sanitize.ts` but is not used).
- Contain the browser with layered controls and add no authentication.

16.4 Decision

Contain the browser in layers: JavaScript disabled, a context-level default-deny route (only `data:`, `blob:`, `about:`), a fresh `BrowserContext` per request, a `p-limit` concurrency limit with a queue cap returning 429, a 20 MB body limit, and timeouts at each level. No authentication. `Compose` binds to `127.0.0.1`.

16.5 Rationale

Comments in `server/pdf.ts`: each control answers a named threat and no single one is load-bearing. The client inlines every asset, so blocking all network access costs nothing legitimate. `docker-compose.yml` states the app has nothing to authenticate. `tasks/todo.md` records the endpoint as deliberately unauthenticated because it grants nothing beyond printing what the caller supplied.

16.6 Consequences

- Exposure beyond loopback publishes a headless browser. Anything reachable needs external access control and TLS.
- No per-client rate limit, so one client can use all capacity.
- Server-side sanitisation is not applied (printSchema is unused).
- Changes to server/pdf.ts require the adversarial probe.

16.7 Related Components

server/pdf.ts, server/index.ts, docker-compose.yml, tests/e2e/adversarial.mjs, SECURITY.md

Chapter 17

ADR-004: Composite string sort key for reading order

17.1 Status

Accepted

17.2 Context

A dropped folder has no defined order. Authors use frontmatter order, numeric prefixes, README/index files and nested directories.

17.3 Options Considered

- A cascade of comparator functions.
- One composite string key per document, sorted with a plain string compare.

17.4 Decision

Each document gets a dot-joined key: `dirRank.explicitOrder.indexRank.numericPrefix.naturalOrder`. Numbers are zero-padded. Missing values sort last within their rank. Ties break on path.

17.5 Rationale

From `src/ingest/ordering.ts`: this makes ordering deterministic and easy to test, stable under equal keys, and a bug is visible by printing the key. Index names outrank numeric prefixes so a folder opens on its README rather than below its chapters.

17.6 Consequences

- Precedence is fixed by the key layout, so changing it means editing `computeSortKey`.

- Numbers are padded to eight digits, and out-of-range values are not handled: uncertain — verify with developer.
- Sorting uses `localeCompare` on the key.

17.7 Related Components

`src/ingest/ordering.ts`, `tests/unit/ordering.test.ts`, `DATA_MODEL.md`

Chapter 18

ADR-005: Repair markdown for rendering only, and report every repair

18.1 Status

Accepted

18.2 Context

Document sets are written by several people over time. Heading hierarchies skip levels, files carry several H1s, and list indentation is inconsistent. Rendering these literally gives a wrong structure and an unusable index.

18.3 Options Considered

- Render literally.
- Repair silently.
- Repair for rendering and record each change for the user.

18.4 Decision

A remark plugin normalises structure before HTML conversion and pushes a RepairNote (rule, line, before, after, detail) for each change. The UI lists them. The source file is never modified.

18.5 Rationale

src/pipeline/plugins/remark-repair.ts and src/types/domain.ts: repairing without telling the user is editing their writing behind their back. Five rules exist: heading-skip, duplicate-h1, list-nesting, mixed-markers, unbalanced-emphasis.

18.6 Consequences

- Every new repair rule must emit a note.
- unbalanced-emphasis only reports. mixed-markers normalises list tight/loose spacing, not marker characters, because mdast does not keep marker characters (comment in the plugin).
- A leading H1 that repeats the document title is dropped and reported, since the reader prints the title itself.

18.7 Related Components

src/pipeline/plugins/remark-repair.ts, src/reader/RepairNotice.tsx, DATA_MODEL.md

Chapter 19

ADR-006: Use `webkitGetAsEntry` for folder drops

19.1 Status

Accepted

19.2 Context

The app must accept dropped folders in the major browsers.

19.3 Options Considered

- `DataTransferItem.getAsFileSystemHandle()` (File System Access API).
- `DataTransferItem.webkitGetAsEntry()`.
- A `webkitdirectory` file input only.

19.4 Decision

Use `webkitGetAsEntry()` for drops, with a `webkitdirectory` file list (`walkFileList`) and `flatgetAsFile()` as fallbacks.

19.5 Rationale

`src/ingest/walker.ts`: despite the prefix, the entry API is the only one implemented across Chrome, Firefox and Safari. The File System Access API is Chromium-only.

19.6 Consequences

- `readEntries` returns batches, so it must be drained in a loop, or large folders are truncated.
- Entries must be captured synchronously, before the first `await`.
- The walk stops at the maximum file count.

19.7 Related Components

`src/ingest/walker.ts`, `tests/unit/walker.test.ts`

Chapter 20

ADR-007: Official Playwright runtime image, run as non-root

20.1 Status

Accepted

20.2 Context

The print service needs Chromium and its OS dependencies in the container, and it renders untrusted HTML.

20.3 Options Considered

- Official Playwright image with playwright-core installed separately.
- A slim Node image with playwright install --with-deps chromium.

20.4 Decision

The runtime stage uses `mcr.microsoft.com/playwright:v1.62.1-noble`. playwright-core is pinned to the same version. The image runs as pwuser.

20.5 Rationale

Dockerfile comments: the image ships Chromium and dependencies but not the npm package, and the versions must match or the executable lookup breaks. The image defaults to root, which disables the Chromium sandbox, so the service does not run as root. The slim-base alternative was deferred so the browser was proven working first (tasks/todo.md).

20.6 Consequences

- The image is about 3.63 GB, since three browsers ship and one is used. The slim alternative is a recorded, unstarted follow-up.

- Upgrades must change the pin and the tag together.
- The hardening in docker-compose.yml applies: read-only root, /tmp tmpfs, cap_drop: ALL, no-new-privileges, ipc: host and init: true for Chromium stability.

20.7 Related Components

Dockerfile, docker-compose.yml, package.json, DEPLOYMENT.md

Chapter 21

ADR-008: Self-hosted fonts and no third-party requests

21.1 Status

Accepted

21.2 Context

The HTML export must open correctly with the network disabled, and the print browser is forbidden from making any request.

21.3 Options Considered

- Load fonts from a font CDN.
- Self-host fonts and bundle them.

21.4 Decision

Fraunces, Newsreader and IBM Plex Mono are installed from `@fontsource/*` packages (SIL OFL) and served from the origin. The app makes no third-party requests.

21.5 Rationale

The README ties self-hosting to offline HTML export. `definition_of_done.md` criterion 0.7 requires that `dist/` contains no `fonts.googleapis/gstatic` references. `src/export/standalone.ts` inlines fonts into the export.

21.6 Consequences

- Fonts add to bundle and export size.
- New assets must also be same-origin, or the export will silently lose them (`collectCss` skips unreadable cross-origin sheets).

21.7 Related Components

`src/styles/fonts.css`, `package.json`, `src/export/standalone.ts`

Chapter 22

Changelog

All meaningful changes to this project are recorded here. Format follows [Keep a Changelog](#). Git history is not a substitute.

22.1 [Unreleased]

22.1.1 Added

- Engineering documentation set under docs/ (architecture, design, workflows, data model, API, security, deployment, testing, operations, maintenance, troubleshooting, dependencies) and ADRs 001-008.

22.1.2 Changed

- README.md shortened to the standard sections. Deep material moved into docs/.

22.1.3 Security

22.1.4 Removed